

PULPino 卷积加速器项目细节总结要点

本文基于当前仓库 `C:/ICP1_Phase2_PULPino` 的实际文件审阅形成。审阅时当前 Git 分支为 `main`，跟踪 `origin/main`，未看到本地未提交修改。本文只总结仓库中可以确认的工作内容：卷积加速器 Register Transfer Level (RTL，寄存器传输级) 设计、通过 APB (Advanced Peripheral Bus，先进外设总线) 接入 PULPino System on Chip (SoC，片上系统)、裸机 C 测试程序、综合、Place and Route (PnR，布局布线) 和 PrimeTime Static Timing Analysis (STA，静态时序分析)。凡仓库无法确认的内容均标为“仓库中未确认”。

1 项目定位、工作边界与整体架构

1.1 本章解决的问题

本章明确项目目标、个人工作边界和系统数据流，避免把 PULPino 原平台内容、未实现功能或无关外设误写成项目成果。

1.2 项目目标与边界

本项目的目标是在 PULPino SoC 中设计并集成一个二维卷积加速器，使 CPU (Central Processing Unit，中央处理器) 软件能够通过 Memory-Mapped I/O (MMIO，内存映射 I/O) 访问 APB 地址空间，向加速器写入输入特征图、启动硬件计算、轮询完成状态并读取输出结果；同时完成 RTL 仿真、软件测试程序准备、Genus 综合、Innovus PnR 以及 PrimeTime STA 分析流程。

仓库中可以确认的实现/修改范围如下：

- 卷积核心 RTL: `rtl/CONV/convolution_accelerator.v`、`rtl/CONV/mac.v`、`rtl/CONV/parse_input_row.v`、`rtl/CONV/ROM.v`。
- APB wrapper: `rtl/CONV/apb_conv.sv`，负责把 APB 访问转换为输入加载、启动、输出保存和读回。
- SoC 集成点: `rtl/peripherals.sv` 中例化 `apb_conv_i`，`rtl/includes/apb_bus.sv` 中新增/保留卷积加速器地址范围，`rtl/periph_bus_wrap.sv` 中将 `s_conv_bus` 接入 APB node。
- 软件测试程序: `ConvolutionAccelerator/apb_conv_printf.c` 是与当前 RTL wrapper 地址窗口最一致的 APB 访问版本；`ConvolutionAccelerator/CONV.c` 是纯软件卷积参考程序；`ConvolutionAccelerator/peripheral.c` 包含 filter 写入窗口，但当前 RTL 未实现 APB 可写 filter，因此不能作为当前硬件接口的准确描述。
- 仿真与后仿真框架: `tb/testbench.sv`、`tb/tb_spi_pkg.sv`、`ConvolutionAccelerator/RTL/tb_conv_acc.v`、`post_synthesis/Makefile`、`post_PR/Makefile`。

- 综合脚本和报告: `synthesis_scripts/synt.tcl`、`synthesis_scripts/design_setup_grade5.tcl`、`synthesis_scripts/create_clock.tcl`、`synthesis_scripts/reports/*`。
- PnR 脚本: `PR_scripts/Design_imports/*`、`PR_scripts/scripts_grade5/*.tcl`。
- PrimeTime 脚本和报告: `primitime/scripts_phase2/primitime.tcl`、`primitime/reports/s_grade5_perip/*`。

作为既有平台、仅与本项目发生接口关系的部分包括 RISC-V CPU core、PULPino AXI (Advanced eXtensible Interface, 高级可扩展接口) interconnect、AXI-to-APB bridge、原有 UART (Universal Asynchronous Receiver/Transmitter, 通用异步收发器)、GPIO (General-Purpose Input/Output, 通用输入输出)、SPI (Serial Peripheral Interface, 串行外设接口)、instruction/data SRAM (Static Random Access Memory, 静态随机存储器)、debug/JTAG 等。本文不把这些模块作为本人实现重点展开, 只说明它们如何把 CPU 访问路由到卷积加速器。

1.3 整体控制与数据流

与本加速器直接相关的端到端路径为:

CPU 软件 → AXI 主端访问 → AXI2APB bridge → APB 地址译码
→ `apb_conv wrapper` → 卷积核心 → APB 读回结果

具体控制流和数据流如下:

1. C 程序使用 `0x1A10_3000` 作为卷积加速器基地址。该地址由 `rtl/includes/apb_bus.sv` 中 `CONV_START_ADDR=32'h1A10_3000` 和 `CONV_END_ADDR=32'h1A10_3FFF` 定义。
2. CPU 发起 MMIO 访问后, PULPino 顶层 `rtl/pulpino_top.sv` 中的 AXI interconnect 根据外设地址段 `0x1A10_0000--0x1A11_FFFF` 将访问送入 `rtl/peripherals.sv`。
3. `rtl/peripherals.sv` 内部的 `axi2apb_wrap` 将 AXI transaction 转为 APB transaction, 并送入 `s_apb_bus`。
4. `rtl/periph_bus_wrap.sv` 将 `s_apb_bus` 交给 `apb_node_wrap`, 并用 start/end address 数组把 `0x1A10_3000--0x1A10_3FFF` 译码到 `s_conv_bus`。仓库中的 `apb_node_wrap` 源文件不在本仓库, 而在综合脚本引用的外部 `ips/apb/apb_node/apb_node_wrap.sv`, 因此 APB node 内部译码细节在本仓库中未确认。
5. `rtl/peripherals.sv` 把 `s_conv_bus.paddr[11:0]`、`pdata`、`pwrite`、`psel`、`penable` 连接到 `apb_conv_i`。`wrapper` 只接收 12-bit local offset。
6. 软件先写控制寄存器 offset `0x000`, 使 `wrapper` 从 IDLE 进入 LOAD; 随后向 offset `0x100--0x40F` 连续写入 784 个 8-bit 输入像素。`wrapper` 将这些数据保存到内部寄存器阵列 `ifm_r[0:783]`。
7. `wrapper` 进入 CONV 后, 每个时钟向卷积核心送入一个像素。卷积核心内部固定 ROM 保存 5x5 kernel, 仓库中未实现 APB 可写 kernel。
8. 卷积核心每次输出一行 24 个 21-bit OFM (Output Feature Map, 输出特征图) 结果, `wrapper` 先锁存到 504-bit `ofm_buffer`, 再逐元素写入三个 2048x8 SRAM 宏 `byte0/byte1/byte2`。

9. 计算完成后，软件读取 status offset 0x004 直到返回 1，再从 offset 0x008 起按 32-bit word 读取 576 个输出结果。每个输出结果实际有效位为 PRDATA[20:0]，高位补零。

注意：卷积输入数据不直接存放在 PULPino 原有 data SRAM 中，而是由 APB wrapper 收到后存入 apb_conv 内部 ifm_r 寄存器阵列；kernel 不由软件加载，而是 rtl/CONV/ROM.v 中固定为全 255；输出结果不写回 PULPino data SRAM，而是写入 apb_conv 内部例化的三个 8-bit SRAM 宏，然后经 APB 读回。

1.4 关键目录树

C:/ICP1_Phase2_PULPino

```
|-- README.md
|-- rtl
|   |-- pulpino_top.sv
|   |-- pulpino_padstop.v
|   |-- peripherals.sv
|   |-- periph_bus_wrap.sv
|   |-- axi2apb_wrap.sv
|   |-- includes
|       |-- apb_bus.sv
|       |-- axi_bus.sv
|       |-- config.sv
|   |-- components
|       |-- SPHDL100909.v
|       |-- cluster_clock_gating.sv
|   |-- CONV
|       |-- apb_conv.sv
|       |-- convolution_accelerator.v
|       |-- mac.v
|       |-- parse_input_row.v
|       |-- ROM.v
|-- Convolution Accelerator
|   |-- apb_conv_printf.c
|   |-- apb_conv.c
|   |-- CONV.c
|   |-- peripheral.c
|   |-- RTL/tb_conv_acc.v
|   |-- MATLAB_stimuli
|       |-- IFM.m
|       |-- IFM.txt
|       |-- OFM.txt
|-- tb
```

```
|  |-- testbench.sv
|  |-- tb.sv
|  |-- tb_spi_pkg.sv
|  `-- uart.sv
|-- post_synthesis
|  |-- Makefile
|  |-- build_libs.sh
|  `-- slm_files/spi_stim.txt
|-- post_PR
|  |-- Makefile
|  |-- do_simulation.tcl
|  `-- slm_files/stimuli/spi_stim_apb_conv_printf.txt
|-- synthesis_scripts
|  |-- synt.tcl
|  |-- design_setup.tcl
|  |-- design_setup_grade5.tcl
|  |-- create_clock.tcl
|  `-- reports
|-- PR_scripts
|  |-- Design_imports
|  |   |-- Import_Design
|  |   |-- LU_pads.io
|  |   `-- my_MMMC.view
|  `-- scripts_grade5
|      |-- 1_floorplan.tcl
|      |-- ...
|      `-- 10_export.tcl
`-- primetime
    |-- scripts_phase2/primetime.tcl
    `-- reports_grade5_perip
        |-- timing_setup.rpt
        |-- timing_hold.rpt
        |-- timing_violation.rpt
        |-- timing_skew.rpt
        `-- power.rpt
```

2 卷积加速器 RTL 微结构

2.1 本章解决的问题

本章说明卷积核心本身如何组织输入、kernel、输出和控制时序，并区分核心 RTL 与 APB wrapper 中的数据缓存。

2.2 关键文件

- rtl/CONV/convolution_accelerator.v: 卷积核心顶层，参数化输入宽度、IFM 尺寸和 kernel 尺寸。
- rtl/CONV/mac.v: 单个 5x5 Multiply-Accumulate (MAC, 乘加) lane。
- rtl/CONV/parse_input_row.v: 将逐像素输入解析成一行 28 像素。
- rtl/CONV/ROM.v: 固定 5x5 kernel ROM。
- rtl/CONV/apb_conv.sv: 不是卷积算子本身，但负责向核心提供逐周期 i_pixel，并保存核心输出。

2.3 顶层模块职责、参数和端口

convolution_accelerator 的默认参数为 PX_W=8、IFM_W=28、KE_W=5。其输入端口为 i_clk、低有效 i_rstn 和单像素输入 i_pixel[7:0]；输出为 o_valid_ofm 和 o_ofm[503:0]。输出宽度来自

$$(2 \times PX_W + \lceil \log_2(KE_W^2) \rceil) \times (IFM_W - KE_W + 1) = 21 \times 24 = 504.$$

因此每次 o_valid_ofm 有效时，核心输出一整行 24 个 21-bit OFM 像素。

核心不直接暴露 APB 信号，也不直接访问 SoC SRAM。它只接收 wrapper 逐周期送来的 8-bit 像素流，kernel 来自内部 ROM，OFM 行结果通过 o_ofm 返回给 wrapper。

2.4 支持的卷积维度、位宽和数值策略

仓库中可确认的卷积规格如下：

- IFM (Input Feature Map, 输入特征图): 28x28, 8-bit unsigned。
- Kernel: 5x5, 8-bit unsigned, 固定 ROM, 当前每个系数为 255。
- Padding: 无 padding。
- Stride: 1。
- OFM: 24x24, 因为 $28-5+1=24$ 。
- 单个 MAC 输入窗口: 5 行, 每行 5 个 8-bit 像素, 共 25 个像素; kernel 同样 25 个 8-bit 系数。
- 乘法结果: RTL 中 prod[g] 为 $[2 \times PX_W - 1:0]$, 即 16-bit。注释里写“15-bit”, 但代码实际为 16-bit。

- 累加结果: `sum_c[20:0]`, 21-bit。最大值 $25 \times 255 \times 255 = 1625625$, 小于 $2^{21} = 2097152$, 因此在当前 8-bit、5x5、全 255 kernel 规格下 21-bit 足够覆盖最大无符号结果。仓库中未实现额外饱和或舍入逻辑, 也未实现将结果截断到 8-bit/16-bit。

2.5 数据存储结构

输入像素在 `wrapper` 内先存入 `ifm_r[0:783]`。进入 CONV 状态后, `wrapper` 每个时钟把一个 `ifm_r` 元素送到核心 `i_pixel`。核心内部 `parse_input_row` 用移位寄存器聚合 28 个像素并产生 `o_valid_row`, 随后 `convolution_accelerator` 把已解析的行保存到 `ifm_r0` 到 `ifm_r27` 这些 224-bit 行寄存器中。

Kernel 由 ROM 根据 3-bit `addr` 输出一行 40-bit 系数。核心通过 `ker0` 到 `ker4` 保存五行 kernel。当前 `rtl/CONV/ROM.v` 的五个有效地址均返回 `40{1'b1}`, 即每行 5 个 `8'hFF`。

输出在核心中按一行 24 个结果并行形成。`wrapper` 在 `ofm_valid` 有效时把 `o_ofm[503:0]` 锁存到 `ofm_buffer`, 再按 21-bit slice 逐个写入三个 SRAM 宏:

- `byte0` 保存结果 `[7:0]`;
- `byte1` 保存结果 `[15:8]`;
- `byte2` 保存 `{3'b000, result[20:16]}`。

这三个 SRAM 均为 `rtl/components/SPHDL100909.v` 中的 `ST_SPHDL_2048x8m8_L` 宏实例, 由 `rtl/CONV/apb_conv.sv` 例化为 `byte0`、`byte1`、`byte2`。

2.6 地址生成和索引规则

核心级地址生成主要是行/窗口选择:

- `parse_input_row` 逐时钟移入一个 8-bit 像素, 按计数器产生行有效信号。代码中当 `cnt==5'h1C` 时产生 `o_valid_row`。
- `valid_row_cnt` 记录已解析行数。行数达到 5 以后, 组合 `case` 逻辑从 28 个行寄存器中选择连续 5 行作为卷积窗口的纵向位置。
- 水平方向由 `generate for (i=0; i<24; i=i+1)` 生成 24 个并行 MAC lane。第 `i` 个 lane 选择每行 `rowX[8*i+39:8*i]`, 即从第 `i` 列开始的 5 个连续像素。
- `wrapper` 保存 OFM 时, `ofm_ptr_r` 从 0 到 23 选择 `ofm_buffer[ofm_ptr_r*21 +: 21]`, `save_cnt` 从 0 到 575 作为三字节 SRAM 的地址。

需要注意的是, `rtl/CONV/parse_input_row.v` 中的移位表达式为 `row[PX_W*IFM_W-5:0]` 与 8-bit 像素宽度不完全匹配; 综合器会按目标位宽处理该表达式, 但仓库中未见对此写法的解释或修正。本文不把它描述成理想化的、已证明无误的 8-bit 整字节移位实现。

2.7 运算数据通路

核心的运算数据通路可以概括为：

像素流 → 行解析 → 5 行窗口选择
→ $24 \times (25 \text{ 个 } 8 \times 8 \text{ 乘法} + 21\text{-bit 累加})$
→ 一行 OFM

每个 MAC lane 在 `i_start_mac` 为高时，把 25 个 IFM 像素和 25 个 kernel 系数装入 `row_r[0:24]` 与 `ker_r[0:24]`。随后 25 个 `prod[g]=row_r[g]*ker_r[g]` 并行组合产生，`sum_c` 在组合 `always @(*)` 中循环累加 25 个乘积。

综合报告 `synthesis_scripts/reports/datapath_pulpino_padstop.rpt` 中可以看到 `apb_conv_i/conv_init/inst_mac[*]` 下的 unsigned 8x8 multiplication 和 `csa_tree_add_69_27_l25_groupi`，说明综合器将 MAC 累加实现成乘法与 Carry-Save Adder (CSA，进位保存加法器) 相关结构。

从吞吐角度看，当前实现是“行内 24 个输出并行、行间随输入流推进”。当已经缓存到足够 5 行数据时，核心每解析出新一行输入就触发 24 个 MAC lane，同一时刻生成该输出行的 24 个卷积窗口结果。MAC 内部没有显式多级流水的加法树寄存器，`o_ofm` 是组合求和结果，`o_valid_ofm` 由 5-bit shift register 延迟 `i_start_mac` 产生。

2.8 控制路径与复位

卷积核心内部没有命名的全局 FSM (Finite State Machine，有限状态机)，主要由计数器和有效信号驱动：

- Kernel 读取计数器 `addr` 在 0--4 循环，驱动 ROM 输出五行系数。
- `parse_input_row` 的 `cnt` 产生 `o_valid_row`。
- `valid_row_cnt` 统计有效行，达到第 5 行后允许 `start_mac`。
- `parsed_a_row && valid_row_cnt_buff>=5` 组合产生 `start_mac`。
- 24 个 MAC 的 `o_valid_ofm` 全部为 1 时，核心顶层输出 `o_valid_ofm=1`。

复位方面，核心使用低有效 `i_rstn`。wrapper 中的 `conv_rstn` 仅在 CONV 状态拉高，因此卷积核心在 IDLE、LOAD 和 READ 状态均被保持复位。仓库中可确认大多数 kernel、计数器和行寄存器在复位下被清零；但 `rtl/CONV/convolution_accelerator.v` 的行寄存器复位拼接中未包含 `ifm_r15` 到 `ifm_r19`，因此这些寄存器的复位值在仓库中未确认。

2.9 可综合 RTL 风格和关键路径

核心主要采用同步时序逻辑保存状态、寄存器阵列和计数器，组合逻辑完成窗口选择、next-state 计算和 MAC 累加。可综合性相关观察如下：

- 24 个 MAC lane 并行实例化，面积主要来自 24 组 25 个 8x8 乘法器和累加树。
- `mac.v` 中乘法和 25 项累加为组合逻辑，可能形成加速器内部主要数据关键路径。

- `synthesis_scripts/reports/area_pulpino_padstop.rpt` 显示 `apb_conv_i` cell area 约 841294, `conv_init` 约 698654, 单个 `inst_mac[*]` 约 23779–23891, 说明 MAC 阵列是面积主因。
- `apb_conv.sv` 使用三个实例化 SRAM 宏保存输出, 而不是推断 block RAM。
- 若从代码质量角度看, 部分时序 `always @(posedge)` 中使用 blocking assignment, 且存在前述复位/移位表达式不一致点; 本文只按仓库实际 RTL 描述, 不把这些写法扩展成未确认的优化设计。

2.10 模块与存储体总结表

模块/寄存器或存储体	作用	读写时机	关键文件
<code>apb_conv</code> <code>ifm_r[0:783]</code>	保存 28x28 IFM, 每项 8-bit	LOAD 状态下 APB 写 offset 0x100--0x40F 时写入; CONV 状态每周周期读出一个像素给核心	<code>rtl/CONV/apb_conv.sv</code>
<code>parse_input_row.row</code>	将逐像素输入聚合成 28 像素行	每个 <code>i_clk</code> 移入 <code>i_pixel</code> ; 计数到代码中的 5'h1C 后产生行有效	<code>rtl/CONV/parse_input_row.v</code>
<code>ifm_r0--ifm_r27</code>	核心内部 28 行 IFM 行缓存, 每行 224-bit	<code>reg_valid_row</code> 有效时按 <code>valid_row_cnt</code> 写入; 窗口选择组合逻辑读取连续 5 行	<code>rtl/CONV/convolution_accelerator.v</code>
ROM 与 <code>ker0--ker4</code>	固定 5x5 kernel, 每个系数 255	ROM 地址 0--4 循环读取; 核心寄存五行 kernel	<code>rtl/CONV/ROM.v</code> 、 <code>rtl/CONV/convolution_accelerator.v</code>
<code>inst_mac[0:23]</code>	24 个并行 MAC lane, 每个计算一个水平输出位置	<code>start_mac</code> 有效时锁存 5x5 窗口和 kernel; 组合乘加产生 21-bit 输出; <code>valid</code> 延迟 5 周期	<code>rtl/CONV/mac.v</code>
<code>apb_conv.ofm_buffer</code>	暂存核心输出的一行 24 个 21-bit OFM	<code>ofm_valid</code> 有效时写入; 随后按 21-bit slice 读出写 SRAM	<code>rtl/CONV/apb_conv.sv</code>
<code>byte0/byte1/byte2</code> SRAM	保存 576 个 OFM, 每个结果拆成 3 字节	CONV 中 <code>save_start</code> 时逐元素写; READ 中按 APB 地址读	<code>rtl/CONV/apb_conv.sv</code> 、 <code>rtl/components/SPHDL100909.v</code>

3 APB Wrapper 与 SoC 集成设计

3.1 本章解决的问题

本章解释加速器作为 APB slave 接入 SoC 的实际层次、信号、地址译码、寄存器映射和软件可见接口。这是本项目硬件集成的核心。

3.2 关键文件

- rtl/includes/apb_bus.sv: APB interface、地址宏和 CONV_START_ADDR/CONV_END_ADDR。
- rtl/periph_bus_wrap.sv: 将 APB 主访问按地址段分发到各 slave; 与本文相关的是 CONV 分支, 其他分支包括 UART/GPIO/SPI/event/FLL/SoC control/debug。
- rtl/peripherals.sv: 例化 axi2apb_wrap、periph_bus_wrap 和 apb_conv_i。
- rtl/axi2apb_wrap.sv: 连接 AXI slave 到 APB master, 实际协议转换模块来自外部 ips/axi/axi2apb。
- rtl/CONV/apb_conv.sv: 卷积加速器 APB slave wrapper。

3.3 Wrapper 的职责与层次关系

apb_conv 存在的原因是卷积核心本身只有像素流输入和 OFM 行输出, 不认识 APB 协议, 也没有软件可见寄存器。wrapper 承担四类职责:

1. 把 APB 写控制寄存器转换成内部 FSM 启动条件;
2. 把 APB 逐字节 IFM 写入转换成内部 ifm_r buffer 装载;
3. 在 CONV 状态向卷积核心逐时钟提供像素, 并收集核心输出;
4. 把 OFM 写入内部三字节 SRAM, 并把 APB 读请求转换成 SRAM 地址和 PRDATA。

层次连接关系为:

```
pulpino_padstop
`-- pulpino_top_i
    |-- axi_interconnect_i
    `-- peripherals_i
        |-- axi2apb_i
        |-- periph_bus_i
        |   `-- apb_node_wrap_i (external IPS source)
        `-- apb_conv_i
            |-- conv_init
            |   |-- inst_parse_input_row
            |   |-- inst_rom
            |   `-- inst_mac[0:23]
            |-- byte0
            |-- byte1
            `-- byte2
```

地址关系如下:

- SoC 外设总地址段: rtl/pulpino_top.sv 中 AXI interconnect 将 0x1A10_0000--0x1A11_FFFF 路由到 peripherals_i。

- APB 外设译码: rtl/includes/apb_bus.sv 定义 CONV_START_ADDR 为 0x1A10_3000, CONV_END_ADDR 为 0x1A10_3FFF。
- 本 wrapper 内部 offset: rtl/peripherals.sv 只把 s_conv_bus.paddr[11:0] 连接给 apb_conv.PADDR, 因此 wrapper 中的 PADDR 是 4 KiB local offset。

3.4 APB 协议信号解释

当前 wrapper 端口没有命名为 PCLK/PRESETn, 而是使用 HCLK/HRESETn 作为 APB slave 时钟和低有效复位。接口信号含义如下:

信号	方向	有效阶段	本项目中的使用方式
HCLK	输入	全周期	wrapper、内部 SRAM 和卷积核心共用时钟。在 rtl/peripherals.sv 中连接为 clk_int[4]。按 APB 语义可对应 PCLK。
HRESETn	输入	异步低有效	复位 wrapper FSM、指针、状态标志、ofm_buffer、PREADY_rst 等。按 APB 语义可对应 PRESETn。
PSEL	输入	SETUP/ACCESS	APB slave 被地址译码选中。wrapper 所有读写使能都要求 PSEL=1。
PENABLE	输入	ACCESS	区分 SETUP 和 ACCESS。wrapper 只在 PSEL && PENABLE 时接受写入或登记读请求。
PADDR [11:0]	输入	SETUP/ACCESS	local offset。控制寄存器 0x000, status 0x004, OFM 读窗口 0x008--0x907, IFM 写窗口 0x100--0x40F。
PWRITE	输入	SETUP/ACCESS	1 表示写, 0 表示读。写 IFM 和启动要求 PWRITE=1; 进入 READ 的完成读触发要求 !PWRITE。
PWDATA [31:0]	输入	ACCESS 写	控制寄存器要求 PWDATA==32'd1。IFM 写时根据 PADDR[1:0] 从 PWDATA 的对应 byte lane 取 8-bit 像素。
PRDATA [31:0]	输出	ACCESS 读	READ 状态下 offset 0x004 返回 1; OFM 读返回 {8'd0, rdata2, rdata1, rdata0}。其他情况默认为 0。
PREADY	输出	ACCESS	大多数状态固定为 1; READ 状态下第一次 OFM SRAM 读 ACCESS 可能拉低一拍, 用 PREADY_rst 为 SRAM 一拍读延迟插入 wait state。
PSLVERR	输出	ACCESS	当前代码固定为 1'b0, 未实现非法地址或非法访问错误上报。
PPROT/PSTRB	无端口	不适用	当前 APB interface 未包含这些信号。wrapper 不能依据 PSTRB 判断 byte enable, 而是通过 PADDR[1:0] 选择 PWDATA byte lane。

APB 两阶段传输在本项目中的语义为:

1. SETUP phase: PSEL=1、PENABLE=0。地址、写方向和写数据已由 master 驱动, 但 wrapper 不在该阶段更新 IFM buffer 或读请求寄存器。

2. ACCESS phase: PSEL=1、PENABLE=1。写事务在该阶段由 wrapper 采样；读事务在该阶段根据 PREADY 决定何时返回有效 PRDATA。

关于 APB back-to-back transfer: APB 本身没有 burst, 本项目 wrapper 支持普通连续单笔 APB transfer。写路径在 LOAD 状态 PREADY=1, 因此 CPU 可以连续发起 IFM byte write, 每一笔 ACCESS 被 wrapper 接收一次。读路径由于 OFM 存在 SRAM 一拍读延迟, wrapper 在 READ 状态利用 PREADY_rst 让每个新读访问的第一个 ACCESS 插入一个 wait state; 因此 OFM 读不是零等待的背靠背读, 而是符合 APB ready/等待机制的多拍读。

3.5 Wrapper 写使能、读使能与地址 decode

写使能 启动写条件为:

```
PSEL && PENABLE && PWRITE && PADDR==12'd0 && PWDATA==32'd1
```

满足该条件时, next state 从 IDLE 进入 LOAD。IFM 数据写条件为:

```
state==LOAD && PSEL && PENABLE && PWRITE && 12'h100 <= PADDR <= 12'h40F.
```

wrapper 并不使用 PADDR 计算 ifm_r 下标, 而是用内部 ifm_ptr 顺序递增。因此软件必须按期望顺序连续写入 784 个 byte; 随机写或漏写不会产生错误响应, 但会破坏 IFM 顺序。写入的 byte lane 由 PADDR[1:0] 决定:

- 00: 写 PWDATA[7:0];
- 01: 写 PWDATA[15:8];
- 10: 写 PWDATA[23:16];
- 11: 写 PWDATA[31:24]。

读使能 OFM 读请求只在 READ 状态登记:

```
PSEL && PENABLE && 12'h008 <= PADDR < 12'h908.
```

ofm_read 是时序寄存器, 用于指示上一拍访问为 OFM 读。SRAM 地址组合计算为:

```
sram_addr = PADDR[11:2] - 10'd2.
```

因此 offset 0x008 对应 OFM index 0, offset 0x00C 对应 index 1, 最后一个按 32-bit 访问的 offset 0x904 对应 index 575。

PRDATA 产生方式 PRDATA 在组合逻辑中默认置 0。进入 READ 状态后:

- offset 0x004--0x007 返回 32'd1, 作为软件轮询的 done/status;
- 若 ofm_read 为 1, 则返回 {8'd0, rdata2, rdata1, rdata0};
- 非法或未覆盖 offset 不报错, 通常读到 0。

由于 ofm_read 是寄存后的读请求, wrapper 用 PREADY wait state 对齐 SRAM 读数据。仓库中未实现独立的 busy 位、ack 清零位或中断输出。

PREADY 和非法访问 PREADY 默认 1。在 CONV 状态也固定为 1；在 READ 状态，若 PREADY_rst==0 且当前为 ACCESS，则 PREADY=0，下一拍 PREADY_rst 被置 1 后 PREADY=1。非法地址不会使 PSLVERR 置位，因为 PSLVERR 被固定为 0。非法写也不会改变状态或报错。

reset 后状态 复位后 wrapper 状态为 IDLE，ifm_ptr、load_done、ifm_input_done、save_start、save_cnt、ofm_ptr_r、save_done、ofm_buffer、ofm_read 和 PREADY_rst 被清零。代码未清零 ifm_r 数组本身，也未清零三个 SRAM 宏内容；这些存储体复位值在仓库中未确认。

3.6 寄存器映射与软件可见接口

寄存器名称	地址 offset / 绝对地址	读写属性	bit 字段	硬件语义	软件使用时机
Control / Start	offset 0x000 absolute 0x1A10_3000	W	PWDATA==1, 实际 C 中用 byte write 写 bit0=1	在 IDLE 状态写 1 后进入 LOAD。读未实现。	软件必须先写 start，使 wrapper 接受后续 IFM 写。
Status / Done	offset 0x004 absolute 0x1A10_3004	R	PRDATA[0]=1, 实际返回 32'd1	READ 状态下返回完成。首次完成后的读会触发状态从 CONV 到 READ，下一次读可稳定看到 1。	软件轮询直到返回 1。无 busy 位、无 clear 位。
OFM read window	offset 0x008--0x907 absolute 0x1A10_3008--0x1A10_3907	R	PRDATA[20:0] 有效，高位补零	按 32-bit word 读取 576 个结果。local offset 0x008+4*i 对应 SRAM index i。	done 后读输出矩阵；C 中 OFM_ADDR[i] 访问。
IFM write window	offset 0x100--0x40F absolute 0x1A10_3100--0x1A10_340F	W	每次有效写取一个 byte lane	LOAD 状态下顺序写入 ifm_r[ifm_ptr]，累计 784 次后 load_done=1 并进入 CONV。	start 后连续写入 28x28 个 8-bit 像素。
Kernel / Filter	仓库中无 APB offset	无	无	当前 kernel 固定在 rtl/CONV/ROM.v，五行均为 8'hFF 系数。	软件不可配置。
Interrupt / Ack / Clear	仓库中无 APB offset	无	无	apb_conv 没有 irq_o 端口，PSLVERR 固定 0，也没有 done clear/ack。	软件只能轮询 status。

4 从 CPU C 程序到硬件执行的完整流程

4.1 本章解决的问题

本章按时间顺序说明裸机 C 程序如何驱动 APB wrapper，以及仿真框架如何装载并执行该程序。重点是软件-硬件契约，而不是逐行翻译 C 代码。

4.2 关键文件

- ConvolutionAccelerator/apb_conv_printf.c: 与当前 RTL 地址窗口最匹配的 APB 测试程序。
- ConvolutionAccelerator/apb_conv.c: 只写中心 20x20 IFM，未写满 784 个像素；按当前 wrapper 会导致 load_done 无法成立，因此不应作为完整测试主程序描述。

- ConvolutionAccelerator/CONV.c: 纯软件 golden/reference 卷积计算。
- ConvolutionAccelerator/peripheral.c: 定义了 filter APB 写窗口, 但当前 RTL 无对应实现, 因此该文件与当前硬件寄存器映射不一致。
- post_PR/slm_files/stimuli/spi_stim_apb_conv_printf.txt: 包含 1A103 相关指令常量的 APB 卷积程序镜像候选。
- tb/testbench.sv、tb/tb_spi_pkg.sv: SPI 装载程序镜像、启动 CPU、等待 GPIO[8] 结束并读取返回码。

4.3 软件初始化与地址定义

ConvolutionAccelerator/apb_conv_printf.c 定义:

- CONV_ACCEL_BASE = 0x1A103000;
- CTRL_REG 为 base offset 0x000 的 volatile uint8_t;
- STATUS_REG 为 base offset 0x004 的 volatile uint32_t;
- IFM_ADDR 为 base offset 0x100 的 volatile uint8_t*;
- OFM_ADDR 为 base offset 0x008 的 volatile uint32_t*。

这些定义与 rtl/CONV/apb_conv.sv 的控制/status/IFM/OFM offset 一致。

4.4 输入数据、kernel 和输出 buffer

ConvolutionAccelerator/apb_conv_printf.c 在 main() 中定义 uint8_t ifm[28][28]={0} 和 uint32_t output[24][24]={0}, 随后把 C 下标 i=4..23、j=4..23 的中心 20x20 区域置为 1。kernel 未由 C 程序写入; 硬件 kernel 固定在 ROM 中, 全 255。输出 buffer 在 C 程序内只是 CPU 读回结果后的存放位置, 不是硬件计算时写入的位置。

4.5 启动、加载、轮询和读回顺序

与当前 wrapper 匹配的顺序是:

1. CPU 写 CTRL_REG=1。该 APB 写使 wrapper 从 IDLE 进入 LOAD。
2. CPU 调用 load_ifm(), 从 IFM_ADDR[0] 到 IFM_ADDR[783] 连续写 784 个 byte。wrapper 每收到一次有效 APB ACCESS 写, 就把相应 byte lane 存入 ifm_r[ifm_ptr] 并递增指针。
3. 当第 784 个 byte 写入后, wrapper 设置 load_done=1, 状态进入 CONV。
4. CONV 状态下 wrapper 释放核心复位, 每个时钟给 convolution_accelerator 一个像素。核心解析行、触发 24 个 MAC lane 并输出 24 行 OFM。
5. wrapper 把每个 21-bit OFM 结果拆成三字节写入 byte0/byte1/byte2 SRAM, save_cnt 达到 576 后置 save_done=1。
6. C 程序在 wait_for_done() 中轮询 STATUS_REG。由于 wrapper 的 READ 状态由一次读访问触发, 完成后的第一次轮询可能只让 FSM 从 CONV 转到 READ, 后续轮询返回 1。

7. C 程序调用 `read_ofm()`，按 `OFM_ADDR[i]` 读取 576 个 32-bit word，保存到 `output[24][24]`。

4.6 PASS/FAIL 与 golden result

仓库中存在纯软件参考程序 `ConvolutionAccelerator/CONV.c`，它使用相同的 28x28 IFM、5x5 全 255 filter，并以 MATLAB `conv2` 的 `valid` 模式等价嵌套循环计算 24x24 OFM。MATLAB 脚本 `ConvolutionAccelerator/MATLAB_stimuli/IFM.m` 也会生成 `ConvolutionAccelerator/MATLAB_stimuli/OFM.txt`。但是，当前 `ConvolutionAccelerator/apb_conv_printf.c` 没有把硬件读回 OFM 与 golden result 自动比较；`print_ofm()` 内的 `printf` 也处于注释状态。因此“硬件输出与 golden 自动比较并 PASS/FAIL”的机制在仓库中未确认。

仿真框架中的 PASS/FAIL 来自 `tb/tb_spi_pkg.sv` 的 `spi_check_return_codes():testbench` 在程序结束后通过 SPI 读取 `0x1A10_7014`，若返回 0 则打印 Test OK，否则打印 Test FAILED。这属于 PULPino 软件返回码机制，不等价于卷积结果逐元素比对。

4.7 端到端对应表

软件操作	APB 操作	硬件动作	关键文件
定义 <code>CONV_ACCEL_BASE</code> <code>0x1A103000</code>	无立即 APB 事务	与 <code>CONV_START_ADDR</code> 对齐，后续 load/store 进入 <code>s_conv_bus</code>	<code>ConvolutionAccelerator/apb_conv_printf.c</code> 、 <code>rtl/includes/apb_bus.sv</code>
<code>start_conv()</code> 写 <code>CTRL_REG=1</code>	写 absolute <code>0x1A10_3000</code> ; local offset <code>0x000</code>	wrapper 从 IDLE 进入 LOAD	<code>rtl/CONV/apb_conv.sv</code>
<code>load_ifm()</code> 连续写 784 byte	写 <code>0x1A10_3100--0x1A10_340F</code>	每笔 ACCESS 写入 <code>ifm_r[ifm_ptr]</code> ；写满后 <code>load_done=1</code>	同上
CPU 继续执行并轮询 status	读 <code>0x1A10_3004</code>	<code>save_done</code> 后读访问推动 wrapper 进入 READ；READ 状态返回 1	同上
wrapper 处于 CONV	无需 CPU 干预	每周向核心送 1 像素；核心按 24-lane MAC 输出 OFM 行	<code>rtl/CONV/convolution_accelerator.v</code> 、 <code>rtl/CONV/mac.v</code>
<code>read_ofm()</code> 读 576 word	读 <code>0x1A10_3008+4*i</code>	wrapper 读三字节 SRAM，插入 wait state 后返回 <code>PRDATA[20:0]</code>	<code>rtl/CONV/apb_conv.sv</code>
程序 <code>return 0</code>	PULPino runtime 写返回码位置，仓库中 runtime 源未包含	testbench 通过 SPI 读 <code>0x1A10_7014</code> 判断返回码	<code>tb/tb_spi_pkg.sv</code>

5 RTL 仿真与 FPGA 验证框架

5.1 本章解决的问题

本章只总结与卷积加速器直接相关的验证文件和流程，并区分核心级 RTL 仿真、SoC 级程序加载仿真、后综合/后布局仿真与 FPGA 上板验证。

5.2 核心级 RTL 仿真

`ConvolutionAccelerator/RTL/tb_conv_acc.v` 是独立卷积核心 testbench。它例化 `convolution_accelerator`，产生 10 ns 周期时钟，先复位 150 ns。

随后 testbench 通过 `$readmemb` 将 `IFM.txt` 读入 memory，共 784 行二进制像素，并在每个 `i_clk` 上升沿把一个像素送到 `i_pixel`。

`ConvolutionAccelerator/MATLAB_stimuli/IFM.m` 生成：

- `ConvolutionAccelerator/MATLAB_stimuli/IFM.txt`: 28x28 输入，中心 20x20 为 1，其余为 0，每行一个 8-bit 二进制字符串；
- `ConvolutionAccelerator/MATLAB_stimuli/OFM.txt`: 5x5 全 255 kernel 的 24x24 参考输出；
- `ofm1.jpg` 和 `ofm2.jpg`: 输出可视化图像。

当前核心级 testbench 没有在 Verilog 中读取 `OFM.txt` 并逐元素 assert，因此可确认的是 stimulus 驱动和波形观察框架；自动 reference comparison 在仓库中未确认。

5.3 SoC 级仿真

`tb/testbench.sv` 例化 `pulpino_padstop`，产生主时钟 `CLK_PERIOD=40ns`，即 25 MHz；也支持 FLL reference clock 配置，但当前参数 `CLK_USE_FLL=0`。testbench 连接 SPI slave model、UART bus、JTAG debug interface，并通过 `gpio_out[8]` 作为程序结束等待信号。

程序装载路径由 `tb/tb_spi_pkg.sv` 实现：

- `spi_load()` 读取 `./slm_files/spi_stim.txt`，用 SPI 写入 instruction/data memory stimulus。
- `spi_check()` 再读回并检查 SPI 写入内容是否一致。
- testbench 通过 JTAG 写 `0x1A10_7008` 设置 boot address 为 `0x0000_0000`，随后拉高 `fetch_enable` 启动 CPU。
- 程序结束时 testbench 等待 `gpio_out[8]`，再调用 `spi_check_return_codes()` 从 `0x1A10_7014` 读取返回码。

仓库中 `post_PR/slm_files/stimuli/spi_stim_apb_conv_printf.txt` 包含 1A103 相关指令常量，说明存在卷积 APB 测试程序镜像候选；但 `tb/tb_spi_pkg.sv` 固定读取 `./slm_files/spi_stim.txt`，仓库中未见自动选择 `spi_stim_apb_conv_printf.txt` 的脚本。因此执行卷积 APB 用例前，需要确认相应 stimulus 是否已复制/链接为当前仿真的 `slm_files/spi_stim.txt`。

5.4 后综合和后布局仿真

`post_synthesis/Makefile` 使用 QuestaSim/ModelSim 流程编译综合后网表 `../Synthesis/outputs/pulpino_padstop.v`，并用 `../Synthesis/outputs/pulpino_padstop.sdf` 做 SDF (Standard Delay Format, 标准延迟格式) 标注。它还通过 `post_synthesis/build_libs.sh` 建立 MEMORY、PADS、CLOCK65LPLVT、CORE65LPLVT 仿真库。

`post_PR/Makefile` 使用 PnR 后网表 `../PR/outputs/pulpino_padstop.v` 和 `../PR/outputs/pulpino_padstop.sdf`，并在 `ANNOTATE_WITH_SDF` 中同时给出 `-sdfmax` 和 `-sdfmin`。它同样编

译 tb/testbench.sv, 并可配合 post_PR/do_simulation.tcl 记录 /export/space/nobackup/fa7855yu-s/VCD_MM.vcd。

5.5 FPGA 或板级验证状态

仓库中没有可确认的 Nexys4 bitstream、下载脚本或完整 FPGA 板级 I/O 约束流程。当前可确认的是:

- ASIC padstop 文件 rtl/pulpino_padstop.v 和 pad placement 文件 PR_scripts/Design_imports/LU_pads.io;
- RTL、后综合、后 PnR 仿真框架;
- PnR layout 图片 PR_scripts/layout.png 和 pulpino_grade5_layout.png。

因此本文表述为“仓库中可确认的是仿真/实现流程, 而非完整 FPGA 上板实测”。

6 综合 (Synthesis) 流程与脚本解读

6.1 本章解决的问题

本章基于当前综合脚本和报告, 说明综合工具、顶层、RTL/file list、库、约束、输出产物及其与 PnR 的关系。同时指出脚本中与卷积文件相关的可复现性问题。

6.2 关键文件和工具线索

- synthesis_scripts/synt.tcl: 主综合流程脚本。
- synthesis_scripts/design_setup.tcl: 基础 design setup, 但当前文件未列入卷积核心 RTL。
- synthesis_scripts/design_setup_grade5.tcl: 包含 rtl/CONV 路径和 apb_conv.sv 的 setup 变体。
- synthesis_scripts/create_clock.tcl: Genus 时钟和 I/O delay 约束。
- synthesis_scripts/reports/qor_pulpino_padstop.rpt、area_pulpino_padstop.rpt、timing_pulpino_padstop.rpt、datapath_pulpino_padstop.rpt: 已有综合报告。

已有报告头部显示: 工具为 Genus(TM) Synthesis Solution 16.12-s027_1; 顶层模块为 pulpino_padstop; technology libraries 包括 CLOCK65LPLVT、CORE65LPLVT、SPHDL100909 和 PAD; operating condition 为 typical。

6.3 RTL source、include path 与库

synthesis_scripts/design_setup_grade5.tcl 设置:

- DESIGN=pulpino_padstop;
- 主要路径变量包括 RTL=\${ROOT}/rtl、INCLUDES=\${ROOT}/rtl/includes 和 COMPONENTS=\${ROOT}/rtl/com

- 卷积与外部 IP 路径变量为 `CONV=${ROOT}/rtl/CONV` 和 `IPS=${ROOT}/ips`;
- HDL search path 覆盖 PULPino RTL、components、APB/AXI/debug/FPU/RISC-V/zero-riscy 等 IPS 目录;
- library search path 指向 ST 65 nm core/clock library、SPHDL100909 SRAM memory library 和 pad library;
- library list 包括 `CLOCK65LPLVT_nom_1.20V_25C.lib`、`CORE65LPLVT_nom_1.20V_25C.lib`、`SPHDL100909_nom_1.20V_25C.lib` 和 `Pads_Oct2012.lib`。

卷积相关文件在 `design_setup_grade5.tcl` 中体现为:

- `DESIGN_FILES_conv` 定义了 `convolution_accelerator.v`、`mac.v`、`parse_input_row.v`、`ROM.v`;
- `DESIGN_FILES_sv6` 中加入了 `rtl/CONV/apb_conv.sv`。

但当前 `synthesis_scripts/synt.tcl` 只 `source design_setup.tcl`, 且即便改为 `source design_setup_grade5.tcl`, 脚本也没有显式 `read_hdl -v2001 ${DESIGN_FILES_conv}`。已有 `area_pulpino_padstop.rpt` 明确包含 `apb_conv_i/conv_init/inst_mac[*]` 层级, 说明报告生成时的综合运行包含了卷积核心; 但是当前仓库脚本的完整可复现性存在缺口。严格复现时应补充读取 `DESIGN_FILES_conv`, 或把四个 Verilog 核心文件加入已被 `synt.tcl` 读取的 file list。该修正意图在仓库中未确认。

6.4 时钟与 I/O 约束

`synthesis_scripts/create_clock.tcl` 使用 Genus 风格命令 `define_clock` 和 `external_delay`:

- 主时钟 `clk`: period 5000 ps, 即 5 ns / 200 MHz; network/source late latency 为 250 ps; setup/hold uncertainty 设置为 250 ps; input/output delay 为 250 ps。
- `spi_clk`: period 50000 ps, 即 50 ns / 20 MHz; latency 和 I/O delay 为 2500 ps。
- `tck_i`: period 50000 ps, 即 50 ns / 20 MHz; latency 和 I/O delay 为 2500 ps。
- 异步时钟组约束使用 `set_clock_groups -asynchronous`, 三个 group 分别为 `clk`、`spi_clk` 和 `tck_i`。

脚本未设置 driving cell、load、multicycle path 或 false path; 注释中保留了跨时钟 false path 示例, 但当前未启用。

6.5 综合主流程

`synthesis_scripts/synt.tcl` 的主要步骤为:

1. 设置 `ROOT`、`SYNT_SCRIPT`、`SYNT_OUT`、`SYNT_REPORT` 并创建输出目录。
2. `source design setup`, 读取 Verilog/SystemVerilog/VHDL file list。
3. 设置 `hdl_error_on_blackbox=true`, 并保留 `hdl_error_on_latch` 注释。

4. elaborate \$DESIGN, 随后 check_design 和 report timing -lint。
5. source create_clock.tcl 施加时序约束。
6. 顺序执行 syn_generic、syn_map、syn_opt。
7. 输出综合网表、SDC 和 SDF: \${SYNT_OUT}/pulpino_padstop.v、.sdc、.sdf。
8. 生成 QoR、area、datapath、messages、gates、timing、timing lint 报告。

6.6 关键报告观察

synthesis_scripts/reports/qor_pulpino_padstop.rpt 显示:

- clocks: clk=5000 ps、spi_clk=50000 ps、tck_i=50000 ps;
- default group critical path slack 为 0.0 ps, TNS 为 0, violating paths 为 0;
- total cell area 约 1717661.546;
- max fanout 为 23681, 位置为 pulpino_top_i/peripherals_i/apb_conv_i/conv_init/inst_mac[3].inst_mac_i/i_clk。

synthesis_scripts/reports/area_pulpino_padstop.rpt 显示 apb_conv_i cell area 约 841294, conv_init 约 698654, 24 个 MAC lane 各约 23.8k cell area。该结果支持“卷积加速器显著影响面积”的结论。

6.7 脚本路径—主要命令—作用—产物—下一阶段关系

脚本路径	主要命令	作用	产物	下一阶段关系
synthesis_scripts/design_setup_grade5.tcl	设置 DESIGN、search path、library、file list	定义综合环境与 RTL/IPS 输入	变量和约束环境	被主综合脚本 source 后供 read_hdl 使用
synthesis_scripts/create_clock.tcl	define_clock、external_delay、set_clock_groups	定义主时钟、SPI 时钟、JTAG 时钟和 I/O delay	综合约束	输出 SDC 供 PnR/STA 使用
synthesis_scripts/synt.tcl	读取 HDL、elaborate、syn_generic/map/opt、输出 netlist/SDC/SDF	完成 RTL 到门级网表综合	Synthesis/outputs/pulpino_padstop.v/.sdc/.sdf 和 reports	PnR 使用网表和 SDC; 后综合仿真使用网表和 SDF

7 PnR (Place and Route) 流程与脚本解读

7.1 本章解决的问题

本章按仓库中实际 PnR 脚本顺序解释设计导入、floorplan、power planning、placement、CTS、routing、post-route optimization、检查、寄生抽取和输出产物。

7.2 关键文件

- `PR_scripts/Design_imports/Import_Design`: 列出 LEF、timing library、RC corner、operation condition 和约束文件。
- `PR_scripts/Design_imports/my_MMMC.view`: Innovus MMMC (Multi-Mode Multi-Corner, 多模式多工艺角) view 定义。
- `PR_scripts/Design_imports/LU_pads.io`: pad placement。
- `PR_scripts/scripts_grade5/1_floorplan.tcl` 到 `10_export.tcl`: 后端实现阶段脚本。

7.3 Design import / netlist import

`PR_scripts/Design_imports/Import_Design` 不是完整的 `init_design` Tcl 脚本, 而是列出导入配置: tech LEF、standard-cell LEF、clock-cell LEF、pad LEF、SPHDL100909 SRAM LEF、MMMC library/RC/op condition 和 SDC 路径 `../Synthesis/outputs/pulpino_padstop.sdc`。仓库中未看到明确的 `init_verilog`、`init_lef_file` 或 `init_design` 命令脚本, 因此自动化 design import 命令在仓库中未确认。

可以确认的输入意图是: 综合后 `../Synthesis/outputs/pulpino_padstop.v` 作为门级网表, `../Synthesis/outputs/pulpino_padstop.sdc` 作为约束, 65 nm LEF/timing/RC files 作为物理和时序库。

7.4 MMMC 设置

`PR_scripts/Design_imports/my_MMMC.view` 定义两组 corner:

- FF: best-case timing library, 1.30V、-40C, RCMIN qrcTechFile, 用于 hold analysis。
- SS: worst-case timing library, 1.10V、125C, RC MAX qrcTechFile, 用于 setup analysis。

脚本创建 Clock_Constraint constraint mode, 读取 `../Synthesis/outputs/pulpino_padstop.sdc`; 创建 FF 和 SS analysis view, 并通过 `set_analysis_view -setup {SS} -hold {FF}` 指定 setup/hold 分析视角。

7.5 Floorplan 与宏放置

`PR_scripts/scripts_grade5/1_floorplan.tcl` 执行:

- `setMultiCpuUsage -localCpu 6;`
- `floorPlan -site CORE`
`-s 3400 3400 30 30 30 30`
`-fplanOrigin llcorner`, 即 3400x3400 core/floorplan 尺寸, 四边 margin 30;
- 放置 data memory bank;
- 用 `dbGet top.insts.name *apb_conv_i/byte*` 找到 `apb_conv_i/byte0..byte2` 三个卷积输出 SRAM 宏, 并相对 core boundary/前一个宏放置;
- 放置 instruction memory banks;

- fit 调整视图。

这说明 PnR 阶段把卷积输出 SRAM 宏作为显式 macro 处理，而不是普通 standard cell。

7.6 Halo、cut rows 和 power planning

PR_scripts/scripts_grade5/2_halo_and_cut_rows.tcl 对所有 block 添加 28 28 28 28 halo，并定义 instruction RAM、data RAM 和 apb_conv_i/byte0..byte2 组成的 conv_banks。随后 selectInst \$all_ram_banks 和 cutRow -selected，为 SRAM macro 周围切 standard-cell row，避免标准单元与宏重叠。

PR_scripts/scripts_grade5/3_global_net_n_power_ring.tcl 执行 VDD/GND global net connection，并创建 core power ring:

- VDD 连接 vdd/VDDC/tiehi;
- GND 连接 gnd/GNDC/tielo;
- addRing 在 core 周围用 M3/M4、width 2、spacing 2、offset 2 创建 VDD/GND ring。

PR_scripts/scripts_grade5/4_power_stripes.tcl 添加 M4 vertical stripe 和 M3 horizontal stripe，set-to-set distance 为 300，width 为 2。该脚本中 source 路径 ./scripts111/3_global_net_n_power_ring.tcl 与当前仓库目录 PR_scripts/scripts_grade5 不一致，属于脚本路径可移植性风险；手动运行时需要修正 source 路径或预先执行第 3 步。

7.7 Placement 与 pre-CTS optimization

PR_scripts/scripts_grade5/5_1_tap_n_std_cell.tcl 设置 placement 目标:

- place_global_uniform_density=true;
- place_global_max_density=0.6;
- congestion effort 和 timing effort 均设为 high;
- 建立 partial placement blockage，density 30，box 为 {118 2399 3500 3500};
- 每 100 距离插入 well tap cell HS65_LH_FILLERNPWFP4;
- placeDesign 完成标准单元放置。

PR_scripts/scripts_grade5/5_2_OPT.tcl 创建 In2Reg、Reg2Out、Reg2Reg path group，设置 setup target slack 0.02，并执行 optDesign -preCTS -setup。这一步把 floorplan/placement 后的网表和约束作为输入，输出 pre-CTS 优化后的放置设计。

PR_scripts/scripts_grade5/5_3_pre-CTS_timing_analysis.tcl 使用 on-chip variation 和 CPPR (Common Path Pessimism Removal，公共路径悲观去除)，分别执行 pre-CTS setup 和 hold timeDesign，输出 timingReports。

7.8 CTS 与 post-CTS optimization

PR_scripts/scripts_grade5/6_1_design_clk.tcl 设置 CTS (Clock Tree Synthesis，时钟树综合) 可用 inverter/buffer cell 和 hold fixing cells，配置 path group effort，生成并 source ccopt.spec，

随后执行 `cchopt_design`。脚本没有手写具体 skew/latency target, 而是依赖 SDC/MMMC 与 `cchopt spec`。

CTS 后优化分为:

- `6_2_Post_CTS_setup_OPT.tcl`: `optDesign -postCTS -setup`;
- `6_3_Post_CTS_hold_OPT.tcl`: 删除 partial blockage, 设置 hold target slack 0.05, 执行 `optDesign -postCTS -hold`;
- `6_4_Post_CTS_setup_OPT_incr.tcl`: 增量 setup 优化;
- `6_5_Post_CTS_hold_OPT_incr.tcl`: 增量 hold 优化。

7.9 Routing、post-route optimization 和检查

`PR_scripts/scripts_grade5/8_1_route.tcl` 先用 `sroute` 连接 VDD/GND 的 block pin、pad pin、core pin 和 floating stripe, 然后设置 NanoRoute 模式并执行 `routeDesign -globalDetail`。当前脚本中 timing-driven routing 和 SI-driven routing 均设置为 false。路由后执行:

- `verify_drc -limit 1000 -report ./reports/verify_drc.pg.rpt`;
- `verifyPowerVia -report ./reports/verify_PowerVia.pg.rpt`;
- `verifyConnectivity -report ./reports/verify_Connectivity.pg.rpt`;
- `checkPlace`。

Post-route 优化由 `8_2_Post_route_setup_OPT.tcl`、`8_3_Post_route_hold_OPT.tcl`、`8_4_Post_route_setup_OPT_incr.tcl`、`8_5_Post_route_hold_OPT_incr.tcl` 完成, 分别针对 setup/DRV、hold、增量 setup 和增量 hold。

7.10 Filler、寄生抽取和输出

`PR_scripts/scripts_grade5/7_filler_cell.tcl` 和 `7_io_filler.tcl` 调整 pad spacing 并加入 IO filler。最终 standard-cell filler 由 `9_filler_cell_last.tcl` 通过 `addFiller` 插入。

`PR_scripts/scripts_grade5/10_export.tcl` 输出:

- `saveNetlist outputs/pulpino_padstop.v`;
- `write_sdf -version 3.0 outputs/pulpino_padstop.sdf`, min view 为 FF, max view 为 SS;
- `rcOut -spf/-spef` 分别输出 `outputs/pulpino_padstop_ss.spf`、`outputs/pulpino_pads_top_ss.spef`、`outputs/pulpino_padstop_ff.spf`、`outputs/pulpino_padstop_ff.spef`。

仓库中未看到 `streamOut`、`saveDef` 或 GDS/DEF 输出命令, 因此 final DEF/GDS 产物在仓库中未确认。

7.11 与卷积加速器相关的物理实现关注点

卷积加速器对 PnR 的影响主要来自:

- 三个 `apb_conv_i/byte*` SRAM 宏需要 macro placement、halo、cut row 和电源连接;
- 24 个 MAC lane 带来大量乘法器和 CSA/adder 逻辑, 面积密度和局部拥塞风险较高;
- `ifm_r` 大型寄存器阵列、`ofm_buffer` 和 SRAM 地址/数据路径会形成较多时序端点;
- `primetime/reports_grade5_perip/timing_violation.rpt` 中出现 `apb_conv_i/byte*`、`ifm_reg`、`ofm_buffer_reg` 等 violation 端点, 说明卷积 wrapper/存储路径是签核阶段需要重点查看的区域之一。

8 PrimeTime STA 与签核检查

8.1 本章解决的问题

本章说明 PrimeTime STA 脚本输入、约束来源、报告含义以及当前报告中可确认的时序状态, 不虚构 WNS/TNS 或不存在的签核结论。

8.2 关键文件

- `primetime/scripts_phase2/primetime.tcl`: 当前 PrimeTime 脚本。
- `primetime_phase1.tcl`: 旧样例脚本, top 为 PADSTOP, memory library 为 SPHD110420, 与当前 `pulpino_padstop/SPHDL100909` 流程不匹配。
- `primetime/reports_grade5_perip/timing_setup.rpt`、`timing_hold.rpt`、`timing_violation.rpt`、`timing_skew.rpt`、`power.rpt`: 与当前卷积集成层级匹配的报告组, 报告中包含 `apb_conv_i`。

8.3 PrimeTime 输入与脚本流程

`primetime/scripts_phase2/primetime.tcl` 设置:

- top design: `pulpino_padstop`;
- power analysis: `power_enable_analysis=true`, `power_analysis_mode=time_based`;
- library search path: `CORE65LPLVT`、`CLOCK65LPLVT`、`SPHDL100909`、`Pads`;
- link path: `CORE65LPLVT_bc_1.30V_m40C.db`、`CLOCK65LPLVT_bc_1.30V_m40C.db`、`SPHDL100909_bc_1.30V_m40C.db`、`Pads_Oct2012.db`;
- netlist: `../PR/outputs/pulpino_padstop.v`;
- SDC: `../Synthesis/outputs/pulpino_padstop.sdc`;
- parasitics: `../PR/outputs/pulpino_padstop_ff.spef`;
- SDF: `../PR/outputs/pulpino_padstop.sdf`, 以 `sdf_max` 读入;
- switching activity: `/export/space/nobackup/fa7855yu-s/VCD_MM.vcd`, strip path 为 `testbench/pulpino_padstop_inst`。

该脚本输出 `report_power`、`min/hold timing`、`max/setup timing`、`report_constraints` 的 all-violators 报告，以及 `clock skew` 报告。

需要注意：PnR MMMC 中 `setup` 使用 SS、`hold` 使用 FF；当前 PrimeTime 脚本 `link path` 和 `parasitic file` 主要使用 FF/bc corner。仓库中未见与 `reports_grade5_perip` 完全对应的多 corner PrimeTime 脚本，因此报告与脚本的一一复现关系需要进一步确认。

8.4 约束含义

PrimeTime 读取的 SDC 来自综合输出，其源头是 `synthesis_scripts/create_clock.tcl`：

- `clk`: 5 ns，用于 SoC 主时钟域，包括卷积 wrapper 所在 `clk_int[4]` 的源时钟；
- `spi_clk`: 50 ns，用于 SPI slave 装载路径；
- `tck_i`: 50 ns，用于 JTAG/debug；
- 三个时钟组声明为 `asynchronous`；
- I/O delay 使用 `external delay` 约束，主时钟 250 ps，SPI/TCK 2500 ps。

仓库中未确认 `false path` 或 `multicycle path` 已被实际启用。

8.5 Setup 与 hold 分析检查内容

Setup analysis 检查数据从 `startpoint` 发出后，能否在目标 `endpoint` 的下一捕获边沿前满足 `setup time`；对应 `report_timing -delay_type max`。Hold analysis 检查数据在同一捕获边沿后是否保持足够长时间，避免太快到达导致旧数据被覆盖；对应 `report_timing -delay_type min`。

阅读报告时应重点查看：

- **Startpoint**: 发起寄存器、memory macro 或 input port；
- **Endpoint**: 捕获寄存器、memory macro pin 或 output port；
- **Path Group 和 Path Type**: 属于 `clk/spi_clk/tck_i` 哪个组，是 `max/setup` 还是 `min/hold`；
- **data arrival time**: 数据实际到达时间；
- **data required time**: 约束要求的最晚/最早时间；
- **slack**: `required` 与 `arrival` 的差值，负值为 violation；
- **clock path、uncertainty、CPR**: 用于判断是否是时钟约束、clock gating 或跨域处理造成的问题。

8.6 当前可确认报告状态

`primitime/reports_grade5_perip/timing_setup.rpt` 显示 PrimeTime 版本为 O-2018.06-SP5，报告日期为 2025-05-26，design 为 `pulpino_padstop`。前 10 条 `setup` 路径中最差 `slack` 为 -4.81 ns，出现在 `debug/JTAG` 相关 `tck_i` 路径；后续多条 `clk` 组路径也有负 `slack`。前 10 条不以 `apb_conv_i` 为主，但 `timing_violation.rpt` 中可见卷积相关 violation 端点，例如 `apb_conv_i/b`

yte2/D[0] slack -0.73 ns、apb_conv_i/ifm_reg[6]/D slack -0.59 ns、apb_conv_i/byte0/A[8] slack -0.42 ns 等。

primetime/reports_grade5_perip/timing_hold.rpt 中前 10 条 hold 路径最差 slack 为 -0.82 ns, 出现在 SPI output path; 另有 clock gating latch 和 data SRAM 到 AXI memory interface 的 hold violation。卷积相关 hold violation 是否进入最差前 10 条在该报告开头未显示, 需结合 timing_violation.rpt 逐项筛查。

primetime/reports_grade5_perip/power.rpt 包含 pulpino_top_i/peripherals_i/apb_conv_i、conv_init 和各 inst_mac[*] 层级, 说明该报告对应包含卷积加速器的 netlist。相比之下, primetime/reports_phase2/power.rpt 中仍出现 apb_timer_i, 与当前 rtl/peripherals.sv 中 apb_conv_i 不一致, 因此本文不用 phase2 报告证明卷积版本 STA。

8.7 综合前、综合后、PnR 后 STA 差异

- RTL/综合前阶段主要依靠理想线延迟或 wireload 估算, 不包含真实布局布线寄生。
- Genus 综合后 timing report 使用综合库和 wireload/enclosed 模式, qor_pulpino_padstop.rpt 中显示 violating paths 为 0, 但该结论不等价于布局后签核。
- PnR 后 PrimeTime STA 读取门级网表、SDF、SPEF 和 VCD, 包含物理布线 RC、cell delay 和活动信息。当前 reports_grade5_perip 显示仍存在 setup/hold violation, 因此不能声称最终时序已完全收敛。

9 项目关键设计选择、工程挑战与可用于面试的要点

9.1 本章解决的问题

本章把仓库中可确认的实现抽象成工程难点、解决方案、验证方式和面试表达要点, 不夸大性能、面积、频率或上板结果。

9.2 关键工程难点

问题一: APB 两阶段时序与 wrapper 状态机对齐 问题: APB 写/读必须在 ACCESS phase 才能被采样, 否则 SETUP phase 的地址和数据可能被误用。实现方案: rtl/CONV/apb_conv.sv 中所有 start、IFM write、OFM read request 均要求 PSEL && PENABLE, 并用 PWRITE 区分读写。为什么这样设计: 符合 APB 两阶段协议, 并避免 SETUP 阶段重复写入。如何验证: 查看 tb/testbench.sv 中 SPI 程序执行路径、ConvolutionAccelerator/apb_conv_printf.c 中 MMIO 操作, 以及波形中 PSEL/PENABLE/PWRITE/PADDR/PWDATA 与 ifm_ptr/state 的关系。

问题二: 软件-硬件地址映射一致性 问题: C 程序中的 base/offset 必须与 apb_bus.sv 和 apb_conv.sv 的 decode 一致。实现方案: 硬件把 0x1A10_3000--0x1A10_3FFF 译码到 s_conv_bus, wrapper 使用低 12 bit offset; apb_conv_printf.c 使用 base 0x1A103000、IFM offset 0x100、OFM offset 0x008。为什么这样设计: 保持 SoC 级地址译码和 slave 内部寄存器映射解耦。如何验证: 用 post_PR/slm_files/stimuli/spi_stim_apb_conv_printf.txt 中 1A103 常量、rtl/includes/apb_bus.sv 地址宏和 rtl/peripherals.sv 连接关系交叉确认。

问题三：IFM 加载顺序与计算启动同步 问题：wrapper 只在 LOAD 状态接受 IFM 写，并且使用内部 ifm_ptr 顺序计数，而不是根据地址随机写入。实现方案：软件先写 start，再连续写 784 个 byte；wrapper 写满后自动进入 CONV。为什么这样设计：简化硬件地址生成，降低 wrapper 复杂度。如何验证：检查 apb_conv_printf.c 的 start_conv(); load_ifm(); wait_for_done(); 顺序，并注意 apb_conv.c 只写中心 20x20，不能触发完整 load_done，因此不是完整验证用例。

问题四：输出保存与 APB 读延迟处理 问题：核心一次输出一行 24 个 21-bit 结果，而软件按 32-bit word 读 576 个 OFM。实现方案：wrapper 用 ofm_buffer 暂存 504-bit 行结果，再用 save_cnt 和 ofm_ptr_r 把每个 21-bit 结果拆成三字节写入 SRAM；读回时用 PREADY_rst 插入 wait state 对齐 SRAM 读延迟。为什么这样设计：将核心宽输出转换成软件易读的 32-bit MMIO 窗口。如何验证：波形中观察 ofm_valid、ofm_buffer、save_cnt、sram_addr、PREADY 和 PRDATA。

问题五：MAC 阵列面积、时序和物理实现压力 问题：24 个并行 MAC lane，每个含 25 个 8x8 乘法和累加，面积和时序压力集中。实现方案：RTL 用 generate 并行展开 24 个 lane；综合器将乘法/累加映射到标准单元和 CSA 结构；PnR 单独放置 wrapper 的三个输出 SRAM 宏。为什么这样设计：以面积换取行内 24 个输出并行，降低控制复杂度。如何验证：综合 area/datapath report 中 apb_conv_i/conv_init/inst_mac[*] 占比，PnR floorplan 中 apb_conv_i/byte* 宏放置，PrimeTime violation report 中卷积相关端点。

问题六：实现脚本与报告的一致性 问题：当前仓库中综合/PnR/PrimeTime 脚本存在路径和版本不一致，例如 synt.tcl source design_setup.tcl 而非 design_setup_grade5.tcl, 4_power_stripe.s.tcl source scripts111, PrimeTime phase2 报告层级与当前 RTL 不一致。实现方案：文档中以包含 apb_conv_i 的 design_setup_grade5.tcl、synthesisreports、PR_scripts/scripts_grade5 和 reports_grade5_perip 为主要证据，同时明确脚本可复现性风险。为什么这样设计：避免把旧报告或不匹配脚本当作最终结果。如何验证：用报告层级搜索 apb_conv_i/conv_init，并检查脚本 file list 和输出路径。

9.3 可用于项目答辩或 RTL/ASIC 面试的要点

1. 本项目不是重新设计整个 PULPino，而是在既有 PULPino SoC 外设地址空间中集成一个 APB slave 卷积加速器。
2. SoC 访问路径是 CPU 发起 AXI MMIO，经过 axi2apb_wrap 转成 APB。随后 periph_bus_wrap/apb_node_wrap 按 0x1A10_3000--0x1A10_3FFF 译码到 apb_conv。
3. wrapper 是核心集成关键：它把 APB 控制/数据事务转换为 IFM buffer 加载、核心启动、OFM SRAM 写入和 APB 读回。
4. 当前 kernel 是 RTL ROM 固定全 255，不是软件可配置；peripheral.c 中的 filter 写接口没有当前硬件支持。
5. IFM 写窗口是 0x100--0x40F，wrapper 使用顺序 ifm_ptr 接收 784 个 byte，不支持任意顺序随机写。

6. 卷积核心支持 28x28 IFM、5x5 kernel、stride 1、no padding，输出 24x24；每个输出是 21-bit，当前规格下足以覆盖最大结果。
7. 数据通路采用 24 个并行 MAC lane，每个 lane 计算一个水平输出位置；核心每次有效输出一行 24 个 OFM。
8. 输出结果先进入 ofm_buffer，再拆成三字节写入三个 SRAM 宏，软件按 32-bit word 读取并获得 PRDATA[20:0]。
9. APB read path 为 SRAM 读延迟插入 wait state，PREADY 不是所有读都零等待；PSLVERR 固定 0。
10. 综合报告显示 apb_conv_i 和 conv_init 是面积大户，MAC 阵列是主要面积来源。
11. PnR 脚本对 apb_conv_i/byte0..byte2 SRAM 宏做了显式 floorplan、halo 和 cut row 处理。
12. PrimeTime reports_grade5_perip 中存在包含卷积端点的 timing violation，因此不能声称最终时序完全签核通过，只能说明完成了 STA 流程并定位了需要关注的路径。

9.4 150–250 字项目描述

本项目基于 PULPino SoC 集成了一个 28x28 输入、5x5 固定 kernel、stride 1、无 padding 的二维卷积加速器。本人完成卷积核心 RTL、APB slave wrapper、SoC 外设地址映射接入和裸机 C 测试程序准备。软件通过 0x1A10_3000 地址段写控制寄存器、连续加载 784 个 IFM byte、轮询完成状态，并从 APB 输出窗口读回 24x24 个 21-bit 卷积结果。硬件侧 wrapper 负责 APB 两阶段事务解析、IFM buffer 管理、核心启动、OFM 行缓存和三字节 SRAM 写回。项目还包含核心级 RTL testbench、SoC SPI 程序加载仿真、Genus 综合、Innovus PnR 脚本和 PrimeTime STA 报告；仓库中可确认完成了实现与分析流程，但未确认完整 FPGA 上板实测或最终无违例时序签核。